

A Live Collaborative Development Platform With CRDT-Based Synchronization

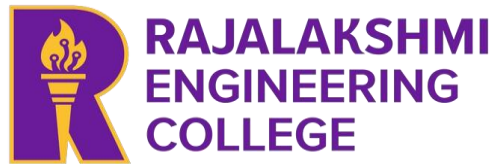
PHASE I REPORT

Submitted by

PRANAAV KUMAR J	230701233
PRANAV RAM S	230701234
RAGHAVAN P S	230701250
PRAVEEN SOMASUNDARAM	230701246

*in partial fulfillment for the award of the degree
of*

**BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE AND ENGINEERING**



**RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI ANNA
UNIVERSITY: CHENNAI 600 025**

December 2026

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this Report titled **“COLABY — A LIVE COLLABORATIVE DEVELOPMENT PLATFORM WITH CRDT-BASED SYNCHRONIZATION”** is the bonafide work of **PRANAAV KUMAR J (230701233), PRAVEEN SOMASUNDARAM (230701246), PRANAV RAM S (230701234), RAGHAVAN P S (230701250)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidates.

“This project addresses the following Sustainable Development Goal(s): **Goal - No 9: Description**. INDUSTRY , INNOVATION AND INFRASTRUCTURE”

SIGNATURE

Dr. J. Manoranjini,
Professor and Head,
Department of Computer Science
and Engineering,
Rajalakshmi Engineering College,
Chennai – 602 105.

SIGNATURE

Dr. S. Madhusudhanan,
Associate Professor,
Department of Computer Science
and Engineering,
Rajalakshmi Engineering College,
Chennai – 602 105.

Submitted to Project Viva-Voce Examination held on

Internal Examiner

External Examiner

ABSTRACT

Software development is becoming more and more collaborative. Developers need to find suitable teammates, communicate with them, manage tasks, and collectively work on a shared codebase. However, in practice, all of these activities are split between many different platforms. One tool is used to find collaborators, another tool is used to communicate, a third tool is used to manage tasks, and so on. Team members cannot work on a single shared project workspace. Traditional video conferencing tools add to the challenge, as they allow other team members to view the code but not work on it collaboratively. ColabY is a concept for an integrated developer collaboration workspace that brings together developer discovery, communities, project creation, management, communication, and collaborative development. The collaborative development workspace is built around Conflict-free Replicated Data Type (CRDT)-based synchronization of the shared code base. Multiple developers can work together on the same codebase utilizing the collaborative editing environment, while their local changes are synchronized with the shared document state. The entire system is implemented as a collection of microservices deployed behind an API Gateway with a React frontend, PostgreSQL and Redis databases, AWS S3 storage, service discovery, and configuration management. An Eclipse Theia-based collaborative editor is used to provide a familiar Integrated Development Environment (IDE) experience rather than a simplistic text editor for collaborative editing. In addition, access to the collaborative editor is controlled at the project level based on user roles. This report describes the problem that ColabY is trying to address, the proposed solution, and the target system architecture. Particular attention is paid to the collaborative development aspect of the system that is built around the CRDT-based synchronization of the shared code repository. Several considerations regarding the development methodology and implementation are discussed and evaluated.

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman Mr. S. MEGANATHAN, B.E, F.I.E., our Vice Chairman Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S., and our respected Chairperson Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D., for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to Dr. S.N. MURUGESAN, M.E., Ph.D., our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to Dr. J. Manoranjini, Ph.D., Professor and Head of the Department of Computer Science and Engineering for his guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, Dr. S. Madhusudhanan, M.E., Ph.D., Associate Professor, Department of Computer Science and Engineering for his valuable guidance throughout the course of the project.

We thank our Project Coordinator, Mr. Karthik, B.E., M.E., Assistant Professor, Department of Computer Science and Engineering for his useful tips during our review to build our project.

PRANAAV KUMAR J : 230701233

PRAVEEN SOMASUNDARAM : 230701246

PRANAV RAM S : 230701234

RAGHAVAN P S : 230701250

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
ABSTRACT.....		3
1. INTRODUCTION		7
1.1 GENERAL		7
1.2 OBJECTIVE		7
1.3 EXISTING SYSTEM.....		8
1.4 PROPOSED SYSTEM.....		8
2. LITERATURE SURVEY		11
2.1 LITERATURE REVIEW		11
2.2 COMPARISON AND DISCUSSION		13
3. SYSTEM DESIGN		15
3.1 SYSTEM ARCHITECTURE		15
3.2.1 SOFTWARE REQUIREMENTS		16
3.2.2 HARDWARE REQUIREMENTS		17
3.2.3 DATABASE REQUIREMENTS.....	Error! Bookmark not defined.	
3.2.4 DEPLOYMENT AND SCALING.....		18
3.3 SYSTEM DESIGN		18
3.3.1 ACTIVITY DIAGRAM.....		19
3.3.2 DATA FLOW DIAGRAM		21
4. METHODOLOGY	Error! Bookmark not defined.	
4.1 METHODOLOGIES		22
4.2 MODULES.....		22
4.2.1 DEVELOPER PROFILE AND DISCOVERY PACKAGE.....		22
4.2.2 COMMUNITY AND PROJECT COLLABORATION PACKAGE		24
4.2.3PROJECT MANAGEMENT AND COMMUNICATION PACKAGE		25
4.2.4 PERSISTENT COLLABORATIVE WORKSPACE PACKAGE	Error! Bookmark not defined.	
4.2.5 REAL TIME CODE SYNCHRONIZATION PACKAGE.....		27

4.2.5 INTEGRATION AND ACCESS CONTROL PACKAGE	27
5. IMPLEMENTATION RESULTS	29
5.1 IMPLEMENTATION RESULTS	29
5.1.1 IMPLEMENTATION OF EMPLOYEE DATA PROCESSING	29
5.1.2 IMPLEMENTATION OF SKILL EXTRACTION AND NORMALIZATION	30
5.1.3 IMPLEMENTATION OF EMPLOYEE RANKING	30
5.1.4 IMPLEMENTATION OF WORKFORCE MATCHING	31
5.1.5 IMPLEMENTATION OF AI-BASED EVALUATION	32
5.1.6 IMPLEMENTATION OF DASHBOARD	32
6. CONCLUSION	33
6.1 CONCLUSION.....	33
6.2 FUTURE WORK	34
REFERENCES	34

CHAPTER 1

INTRODUCTION

1.1 GENERAL

Software development is increasingly collaborative, requiring developers working on the same project to find suitable teammates, communicate effectively, divide responsibilities, manage tasks, and contribute to a shared codebase. In practice, these activities are often distributed across several independent platforms, where a developer may use one service to find collaborators, another for communication, another for project management, another for online meetings, and a separate development environment for writing and managing code.

This fragmented workflow creates frequent context switching and makes it difficult to maintain a single, continuous project workspace. Conventional online meeting and screen-sharing systems further limit collaboration because participants can generally only observe another developer's development environment without directly interacting with or modifying the shared code. To address these challenges,

ColabY is proposed as an integrated developer collaboration platform that brings developer discovery, communities, project creation, project management, team communication, and real-time collaborative coding together within a single environment. Its collaborative development environment is based on CRDT (Conflict-free Replicated Data Type) based synchronization, which enables multiple users to work on a shared codebase in real time while maintaining consistency across participants.

The overall goal of ColabY is to reduce dependence on disconnected development and communication tools and provide a unified environment where developers can discover teammates, connect with others, organize projects, communicate effectively, and build software collaboratively. ColabY is designed to function as both a social collaboration platform for developers and a comprehensive development workspace supporting the complete software development lifecycle from developer discovery and team formation to project creation, project management, team communication, and real-time collaborative coding. system can support both effective project team formation and continuous employee skill development.

1.2 OBJECTIVE

The primary objective of the proposed system is to develop an integrated developer collaboration platform that enables users to find suitable collaborators based on their skills and interests and provides a common environment for creating and managing software projects. The system aims to facilitate effective communication among project members through integrated collaboration features, support developer communities for interaction and knowledge sharing, and enable multiple authorized users to work simultaneously on a shared codebase through real-time collaborative coding.

The collaborative development environment uses CRDT-based synchronization to maintain a consistent shared state during concurrent editing. The system further aims to reduce dependency on multiple disconnected platforms by integrating developer discovery, team formation, project management, communication, and collaborative development within a unified workspace, thereby supporting effective project collaboration and continuous skill development.

1.3 EXISTING SYSTEM

In the existing system, developers rely on multiple independent platforms to support various stages of software development and collaboration. Separate platforms are typically used for finding suitable collaborators, managing projects and tasks, communication with team members, online meetings, and writing and managing code. This distribution of activities creates a fragmented development workflow in which developers are required to switch continuously between different tools and platforms. Project management systems operate separately from coding and communication environments, making it difficult to maintain a common workspace for project activities.

Existing online meeting and screen-sharing platforms provide remote communication and allow developers to share their development environment with other participants. However, the shared development environment is generally controlled by the participant who is sharing the screen, while other members primarily observe the IDE rather than directly interacting with or modifying the code. This limits the level of active collaboration that can be achieved during development.

Furthermore, existing code collaboration solutions may require an active host session for collaboration and do not always provide a seamless mechanism for teams to resume collaborative development later. Parallel modifications can also result in merge conflicts, while communication and project management often require additional external tools.

Hence, the existing system lacks a unified environment that combines developer discovery, community interaction, project management, team communication, and real-time collaborative coding resulting in frequent context switching, limited interactive collaboration, and difficulty in maintaining a continuous project workspace

1.4 PROPOSED SYSTEM

The proposed system, ColabY, is an integrated developer collaboration platform, which unifies the necessary means for engaging and coordinating developers in a shared software development space. The system incorporates developer profiles, skill- and interest-based discovery of developers and communities, project creation and management, team communication, and real-time collaborative coding in a single platform, thus eliminating the need to resort to multiple applications for different stages of the software development cycle and providing the developer with an uninterrupted collaborative workspace.

The system in question would allow developers to create profiles with the relevant skills and interests and use the information to find collaborators. The identified developers can then go ahead and create collaborative projects, add members, organize development, and track issues and tasks within the system in question. Furthermore, the system would provide the communication infrastructure for the members of a project to communicate and synchronize their activities.

The proposed system would incorporate a collaborative coding environment, which is critical for the effective cooperation of developers. Unlike the existing screen-sharing solutions, which basically provide video conferencing means for developers to view the same integrated development environment (IDE), ColabY is positioned as a platform that would allow developers to actively engage in collaborative coding. The collaborative editor would utilize the Conflict-free Replicated Data Types (CRDT) synchronization algorithm to manage simultaneous editing and ensure the consistency of the shared code state between the participants.

The proposed system would be structured as a set of microservices, which would include the authentication infrastructure, developer profiles and discovery, communities, projects, project management, collaboration, code collaboration, real-time communication, and shared infrastructure components. Overall, ColabY is designed to

address the needs of a modern collaborative software development ecosystem and provide an end-to-end solution for developer communities, ranging from finding collaborators and organizing the development process to project management, communication, and real-time development..

.

CHAPTER 2 :

LITERATURE SURVEY

2.1 LITERATURE REVIEW

The literature survey for ColabY includes the studies related to real-time collaborative development, synchronization of shared code, developer collaboration, project community, knowledge exchange and distributed software development. The reviewed articles and research studies present the understanding of collaborative editors, conflict resolution, developer trust, code ownership, community interaction and remote collaboration to highlight the limitations of current solutions and provide theoretical background for the proposed ColabY platform.

S. Steffi Nivedita, P. U. Harsha, P. Rahul, P. Surya Tej and P. Venkat Balaji Naidu (2025) in “A Unified Multi-Modal Real-Time Collaborative Development Environment Integrating Code Editing, Voice Communication, Drawing, and File Synchronization” suggest a collaborative development environment based on CRDT and HLC for code and file synchronization and WebRTC and SFU for voice communication and stroke simplification for collaborative drawing. The developed solution achieved low-latency and conflict-free collaboration with latency of 42-54 ms. However, the article lacks the discussion of developer discovery and social collaboration layer. This literature, thus, helps to identify the need to integrate the features of developer discovery and communities as well as project collaboration and real-time development in ColabY platform.

Xin Tan et al. (2024) in the study of VS Live Share describe the collaborative coding practices and challenges via interviews and a survey among 103 developers. The article underlines the relevance of Live Share collaboration features, especially live editing and conflict resolution. In addition, the researchers suggest that the developers benefit from active coding in shared instance rather than viewing the work space of another developer. This idea can be integrated into ColabY to provide real-time collaborative coding as one of the key features of the platform.

Aparna A. Veer et al. (2025) in “Study on Real-Time Web IDE Collaborative Editors: A Survey Study on Operational Transformation versus Conflict-Free Replicated Data Types” present the survey study comparing two approaches to real-time collaborative editors, namely Operational Transformation and CRDT. The authors identified high convergence for CRDT-based solutions. The study gives justification for selection of CRDT for real-time collaborative coding in ColabY as multiple developers will edit the same files simultaneously.

Hitesh Sapkota et al. (2019) in “Network-Centric Trust Estimation for OSS Developers” assess the trust estimation of open source software developers through subjective-logic-based trust propagation across a large-scale developer network. The article describes how the trust scores impact the likelihood of contributions from certain developers being accepted. In addition, the study offers insights regarding the network structure and how various trust scores interrelate. This literature is key to understand the necessity of developer authenticity and trust within the collaborative development space, specifically to identify trustworthy contributors prior to engaging in collaboration.

Anders Sundelin and Richard (2025) in “Code Ownership and Technical Debt — Modeling Clone Introduction by Teams” analyze code ownership and technical debt with a special focus on clone introduction by development teams. The researchers utilized a Bayesian framework across a large-scale dataset of file edits to evaluate software repositories and teams’ codeownership. The article provides background on how collaboration is essential for code ownership and contribution understanding, which is crucial for ColabY to estimate contributions from different developers within the same project.

Gleiph Ghiotto and Leonardo Murta (2018) in “On the Nature of Merge Conflicts: A Study of 2,731 Open Source Java Projects Hosted by GitHub” analyze merge conflicts across 2,731 open source Java projects hosted on GitHub. The researchers highlight merge conflict occurrences across the projects, which demonstrate that most merge conflicts do not require the introduction of new code but rather reflect the way changes are merged in parallel. The study, therefore, explains the nature of merge conflicts and how they impose limitations on collaborative development, which is a major factor in selecting CRDT-based real-time collaboration in ColabY to avoid merge conflicts.

Ömer Esas and Elisabetta Di Nitto (2022) in “Design Patterns and Anti-Patterns in Microservices Architecture” discuss design patterns and anti-patterns encountered within microservices architecture through the literature review and analysis of several open source applications. The article underlines the importance of individual consideration for all design patterns according to the project requirements. This is a significant point for ColabY as the platform will be constructed from various services, including authentication, developer profiles, project collaboration services, communication and real-time collaborative coding.

Premkumar Devanbu and Vladimir Filkov (2014) in “How Social Q&A Sites are Changing Knowledge Sharing in Open Source Software Communities” analyse the emergence of social QA sites and their impact on knowledge sharing within open source software communities. The researchers demonstrate that knowledge sharing within open source projects has been increasing in recent years due to developers’ engagement with social QA sites. This growth is attributed to the ability of QA sites to enable Q&A interactions that facilitate knowledge sharing and problem solving. This study helps to highlight the importance of QA aspects within the community collaboration features of ColabY.

Andersen and Astri Moksnes Barbala (2023) in “Pair Programming Practiced in Hybrid Work” explore pair programming practices and perspectives in hybrid work settings through interviews, observations and a survey. The study indicates several challenges of hybrid pair programming, including the issue of divided attention in situations when one developer is onsite while another is remote. The article helps to understand the limitations of hybrid collaboration and the need for remote collaboration tools, such as those planned for ColabY to ensure effective pair programming.

Prabhnoor Singh, Adesh Mishra and Dr. Jasleen Kaur (2024) in “Real Time Collaborative Whiteboard” assess real-time collaborative whiteboard application. The developers utilized the collaborative whiteboard tools to demonstrate the benefits of real-time interactions for remote team members. Although the study does not mention collaborative development, it highlights the value of real-time collaboration features for remote team members, which can be applicable to ColabY.

2.2 COMPARISON AND DISCUSSION

The reviewed literature evidence shows that particular individual solutions already address different aspects of software collaboration. The real-time collaborative development environments focus on providing developers with the shared code editing and synchronization tools, whereas the developer community platforms emphasize the interaction and knowledge exchange. The other research explored the concepts of developer trust, ownership, contribution tracking, and remote pair programming. However, most of these solutions address coding collaboration as a separate task and do not consider it as a unified process.

The CRDT and collaborative editors’ research evidence provides the theoretical basis for ColabY’s real-time coding collaboration feature. The existing findings confirm that the simultaneous code editing can be supported by modern collaborative editors using the distributed synchronization algorithm. The analysis of merge conflicts shows that the traditional approaches to parallel programming have considerable limitations. ColabY’s collaborative coding feature addresses this complexity by implementing the CRDT paradigm. Moreover, the tool embeds the collaborative coding technology into a comprehensive platform that incorporates other software development and developer collaboration features.

The reviewed solutions for developer trust establishment and code ownership highlight different techniques for improving the code collaboration mechanisms. The analysis shows that for many developers, finding trusted partners, establishing code contribution rules, and organizing the development team are critical issues. ColabY’s developer profiles,

discovery mechanism, project-member organization tools, and project management features address these requirements.

The findings about community interaction, social knowledge sharing, and remote pair programming emphasize the role of communication and interaction in collaborative software development. ColabY's community engagement and project communication features combine the elements of social coding and team/project management to facilitate collective development tasks.

The key conclusion from the bibliographic research is that the unified developer platform is necessary to address all the critical aspects of software development and collaborative coding. Most existing solutions aim to improve particular elements of developer collaboration, whereas addressing all the requirements remains a challenging scientific and technical task. ColabY claims to provide a comprehensive working environment that integrates modern solutions for developer cooperation.

CHAPTER 3

SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

The proposed ColabY platform is expected to adopt a microservice-based architecture that will organize the developer discovery, community engagement, project management, team communication, and real-time coding collaboration processes into a single system. The architecture is divided into the general description of modules with specific responsibilities, which are organized from the user interface down to the database. The system will utilize an API Gateway as an entry point to the system, where the frontend will communicate with the backend via REST API and WebSocket connections.

The client layer represents the application's Frontend, where developers will interact with the system. It is expected that the Frontend will host developer profiles, discoverable elements, communities, project spaces, project-management tools, communication features, and the collaborative development platform. The client layer will send requests to the API Gateway in order to communicate them to the necessary backend services.

Authentication Service will host all services related to user verification and authorization. The User/Profile Service will manage information about developers and facilitate the discovery of other developers within the system. Community Service will similarly enable community-level interactions. Project Service will host features related to project initiation, project-specific tools, and management of members, roles, invitations, and progress. Project Management Service will provide tools for task management, issues, labels, priorities, and project-wide activity tracking. Collaboration Service will manage collaboration-related activities, including communication and messaging, notifications, and user presence tracking.

The Code Collaboration Service will host the collaborative development platform, which will manage collaborative coding sessions initiated within the system. The collaborative development environment will utilize a CRDT synchronization algorithm to ensure that simultaneous changes made by different developers are correctly merged into the shared code repository. The code collaboration will take place in a development editor, which will provide more functionality than a traditional screen-shared coding session.

PostgreSQL databases, Redis, AWS S3, service discovery, configuration, notifications, and monitoring will comprise the systems' shared infrastructure. Overall, the described architecture will organize the system around a microservice-based pattern that will facilitate individual responsibility for different areas of the system while also enabling close collaboration between them. The project's architecture will define the key components and

their relationships that will comprise the target system.

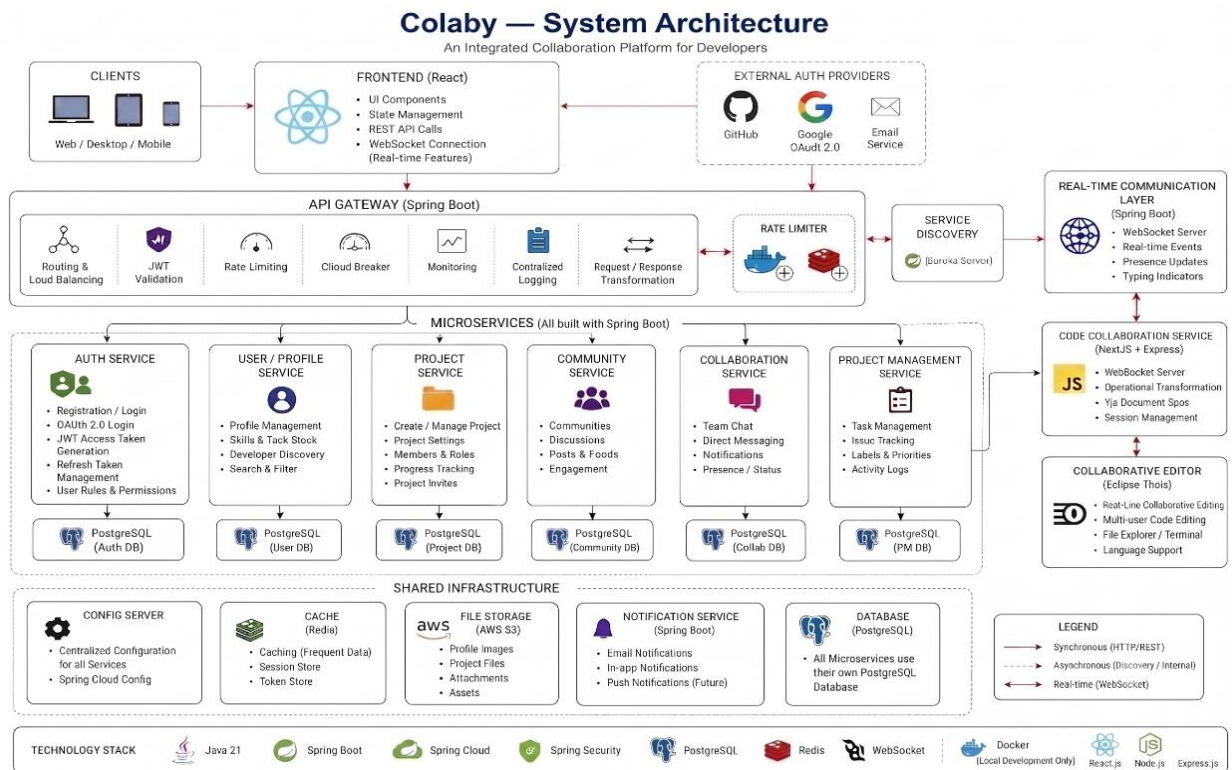


Figure 3.1. Architecture diagram for the entire system

3.2.1 SOFTWARE REQUIREMENTS

The system requirements describe the software, hardware, and infrastructure necessary to develop the software application Colaby. The proposed system involves multiple backend components and processes related to real-time collaboration. Thus, the requirements include both the features related to web apps and specific aspects of real-time collaboration software.

Table 3.1 Software Requirements of the Proposed System

Category	Suggested Version (or) Release	Requirement
Frontend	Latest stable	REACT
Backend	Latest stable	SPRING BOOT
API communication	Latest stable	REST APIs
Real-Time Communication	Latest stable	WEBSOCKET

Authentication	Latest stable	JWT and OAuth 2.0.
Database	Latest stable	PostgreSQL.
Cache/Session Support	Latest stable	Redis.
Synchronization	Latest stable	CRDT
Operating Environment	Latest stable	Windows / Linux
Browser	Latest stable	Modern Chromium-based or equivalent browser
Object Storage	Latest stable	AWS S3.
Configuration	Latest stable	Config server.

3.2.2 HARDWARE REQUIREMENTS

The CPU and memory capacity are important for efficiently processing the employee skill database, generating the semantic embeddings and running the candidate ranking model. The proposed system requires a basic hardware configuration which is suitable for development, testing and deployment of the application.

The system needs a multi-core processor with at least four cores and 16 GB RAM to process employee records, skill representations and model processing. It is advisable to have a minimum of 512 GB of storage to store the employee database, project information, trained models and application files. Optional GPU is available to speed up embedding creation and machine learning. The application components and the API based services need a stable network connection to communicate. Table 3.2 summarizes the recommended hardware configuration.

Table 3.2 Hardware Requirements of the Proposed System

Hardware Components	Suggested Specification	Purpose
CPU	Multi-core processor with more than 4 cores	For supporting efficient data processing, skill extraction and candidate ranking.
RAM	16 GB or higher	For running multiple microservices, database services, development tools, and collaborative sessions simultaneously.
Storage	512 GB or higher	For storing source code, databases, project data, application files, and development dependencies.
Server	Windows or Linux server with sufficient processing and memory	For hosting the backend microservices, databases, API Gateway, real-time collaboration services, and supporting concurrent user requests.
Network	High-speed stable internet connection	For supporting real-time WebSocket communication, API requests, and collaborative code synchronization..

3.2.3 DEPLOYMENT AND SCALING

The distributed design of ColabY makes it possible to scale particular services depending on the load. Those services which are under higher pressure can be allocated more resources than others and, therefore, the overall application will not be scaled up as much.

The API Gateway is used to route requests, while the service discovery component is utilized to identify which instance of a service is available at a certain moment. Redis is applied to reduce the number of calls to the database by caching the results of these calls, and the independent databases of services help decouple the application's domains.

The real-time collaboration component is the most critical one due to the constant exchange of information and the need for a persistent connection for code collaboration. That is why the code collaboration service has to be deployed in such a way that it can withstand the expected number of concurrent sessions and users.

3.3 SYSTEM DESIGN

The system design of ColabY was created that describes the system's significant components and their interactions. The application's frontend is built with React framework that connects to the API Gateway. The said gateway is used to route API requests and responses between the user's browser and backend services. The backend services include Authentication, Users and Developers Directory, Projects, Communities, Teams, Code Collaboration that are organized with PostgreSQL databases as their data storage. The Code Collaboration service uses the CRDTs technology to enable the real-time collaborative coding environment. The Redis database, AWS S3 service, service discovery, configuration service, and notification service are also employed to support caching, object storage, service coordination, centralized configuration management, and notifications, respectively. The overall system design was created in a way that each component serves a specific purpose while the system operates as the united developer collaboration platform.

3.3.1 ACTIVITY DIAGRAM

The main activity flow of ColabY starts with the user's authentication, followed by the developer's discovery process and cooperation. The activity flow depicts the whole intended end-to-end use case of the platform, starting from developer's discovery and leading to project's formation, management, communication, and collaborative development. The project's live-demo workflow also includes the developer's profile creation, discovery, creation of a project, addition of members, communication, and collaborative coding as

sequential stages of the complete use case.

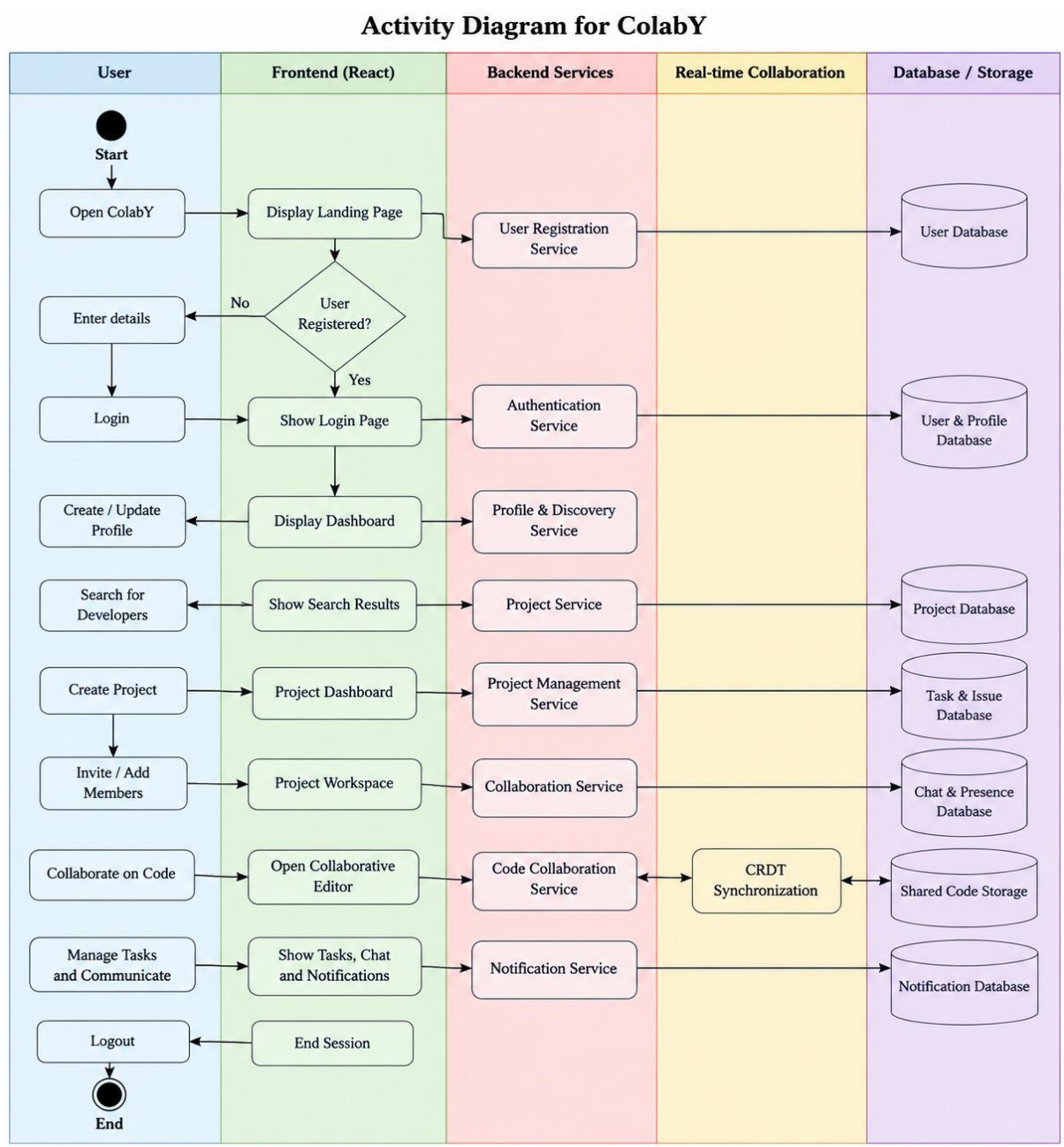


Figure 3.2

3.3.2 DATA FLOW DIAGRAM

This Figure 3.4 illustrates the ColabY's high-level data flow. Data is passed between users, the frontend application (a React client), the backend gateway, backend micro-services, databases and the real-time collaboration layer.

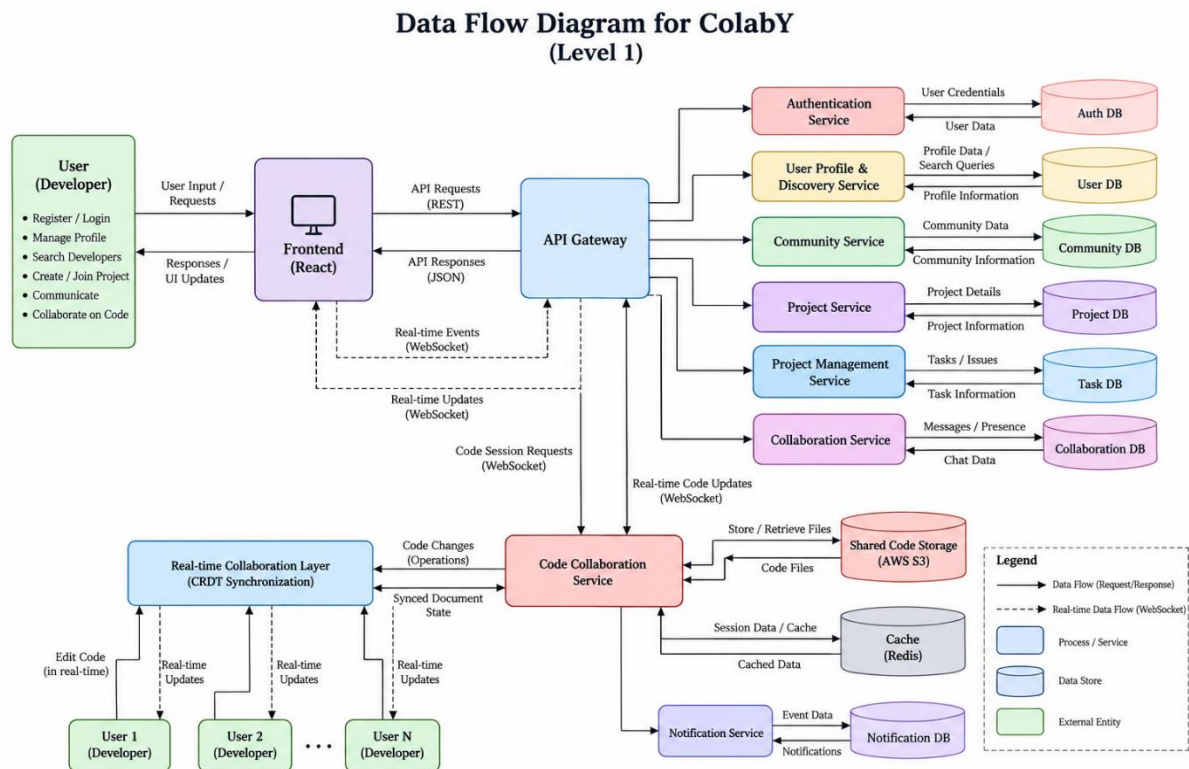


Figure 3.4. Data Flow Level 1 Diagram of the System

The core data transmitted include: authentication tokens, profile data, individual interests and skills, project details, team member data, task and issue information, communication details and notifications, presence data and collaborative code edit messages. Real-time code edits are streamed using the collaboration layer to be synchronized using CRDT.

CHAPTER 4

METHODOLOGY

4.1 METHODOLOGIES

The envisioned ColabY framework is based on a homogenous collaborative software development methodology. The developed methodology encompasses developer discovery, project creation, project management, project collaboration, real-time collaborative development, within the context of a persistent project workspace. A developer account is created comprising specific technical skills, areas of interest and collaboration data with the purpose of facilitating developer discoverability and project team formation.

Developers can either initiate or join a project, which creates a new project entry, persistent project workspace which contains the project data, the environment state, and information about collaboration.

The members of the project can assign themselves and distribute tasks, communicate amongst each other and have access to the development environment from within an Eclipse Theia based IDE. During collaborative development, the users simultaneously manipulate the shared source code with Yjs based CRDT synchronization being used to represent and merge divergent changes and WebSockets used to propagate the real time updates to the users. Development environments can be virtualised and encapsulated by Docker based project environments to ensure that necessary dependencies and environments do not migrate with the individual developer. The persistent workspace makes development feel continuous since collaborators can step away from collaborative development and later re-enter it without the context of development having the properties of a temporary, hosted session.

4.2 MODULES

The ColabY system is broken down into functional packages. An overall system consists of different modules interacting with one another to provide developer discovery, project collaboration, communication, project management and persistent real-time collaborative development. These packages can be grouped in order to implement core functionality which will facilitate an ongoing process of development within the overall system. Development workflow can hence be considered as being based on the following packages:

1. Developer Profile & Discovery Package
2. Community & Project Collaboration Package

3. Project Management & Communication Package
4. Persistent Collaborative Workspace Package
5. Real-time Code synchronization package

There are a number of modules under each of the above packages which is implementing the respective part of the overall ColabY function.

4.2.1 DEVELOPER PROFILE AND DISCOVERY PACKAGE

This package offers the functionality to setup developer identity and provides help to users finding appropriate team mates using their technical interest and abilities. This package represents the first stage of CollabX workflow where developer configures the profile and makes the technical skills and collaboration interest visible for identification of other developers. This information gathered in this package later uses when forming team and looking for team mate.

1. Developer Profile Module: This module enables developer to configure and edit his/her profile which represented the development ability and also describes, the programming knowledge, technology, interest of developer etc and also available in order to find developer from its interest as well as from its skills.
2. Skill Management Module: The technical skills can be maintained from this module so that while we look for specific technique we could find programmer using it, the technical skills includes, languages, programming frameworks, tools, expertise etc.
3. Interest Management Module: This module makes to sure to add all possible interests a programmer can have when collaborating. All the interests will be of technical kind along with developing context, e.g. Web developing, Android developing etc. Along with specific techniques like (java technology used) if any.
4. Developer Search Module: Users can search for registered developer using their name or any related technical aspects from its already registered profile.
5. Developer Filtering Module: A user can filter developers based on interest as well as the skills based upon project need.
6. Collaborator Identification Module: It help the user find collaborator with respect to its profile, using skills, interests or any available information so that new project can be made in collaboration.

This package would set up developer network in ColabY. And we could come from individual developer profile to the collaborator identification using this feature.

4.2.2 COMMUNITY AND PROJECT COLLABORATION PACKAGE

This package makes it possible to utilize the facilities required by developers, with a focus on enabling their participation in the broad community to enable actual software projects. Communities provide opportunities for broader social interaction beyond a project, while the project collaboration modules contain all facilities needed in order to get together and define project teams.

- 1.2 Community Module: In order to get developers, where it is feasible to form a community (on common technical interest). These communities is a broader framework in which a developer could participate in, without it being necessary to share common project affiliations.
- 1.3 Discussion Module: Used to create dialogues between members on the community level. The module allows discussion on technology-related concepts and the sharing of project information, views and experiences.
Post & Feed Module: Enables community members to share their posts, as well as enable other members with interest in specific technology to follow others (feed), through posts and updates regarding technology discussions.
- 1.4 Project creation Module: Used to create shared collaborative software projects. A "project" itself can serve as the organizational element that holds projects members, tasks, files, etc together in a collaborative project.
- 1.5 Project membership Module: Allows the management of the developers linked with an individual project, and the adding or inviting of members. This enables managing the link between a developer and a project.
Role & permission module: This will keep track of the developers linked to projects, and assign relevant roles and corresponding access rights when collaborating.
- 1.6 Project workspace Module: This module enables defining of an individual project's workspace on a workspace level, on where collaborative project activities and shared project resource(s) take place.

The module integrates the social developer layer and the project layer for the collaboration within the different features in ColabY; enabling ColabY's developer the access from community to collaboration.

4.2.3 PROJECT MANAGEMENT AND COMMUNICATION PACKAGE

The package offers the functionalities needed for the management of project tasks as well as for the promotion of human communication in the project team. It bridges together the need for both project management and human interaction by allowing developers to organize responsibilities, monitor project activities and communicate without using additional external applications..

1. Task Management Module: the package should allow project members to generate development tasks and assign it to members, as well as monitor the progress of development tasks. This module should allow breaking the problem to pieces by enabling effective partitioning.
2. Issue Management Module: the package should provide a platform to report and monitor issues detected during the development cycle. This module would allow a precise identification, cataloguing and monitoring of project-related problems.
3. Project Tracking Module: the module must allow the monitoring of current project activities and progress. This module would enable the team members to stay up to date regarding the development status of the project.
4. Team Communication Module: it must provide a system allowing human interaction between project members in a team environment. The aim is to have a team communication layer where developers can work and discuss about development without using a separate chat client or an external communication platform.
5. Direct Messaging Module: individual users should be able to have one-to-one chat interactions to discuss about topics related to one or more project members.
6. Presence Module: a system to keep track of the presence or absence status of users. This would allow each team member to know who is working on current tasks.
7. Notification Module: it should allow users to be notified of changes and events occurring in the project environment.

This package consists in the human communication and management layer of ColabY allowing developers to maintain the task ordering and the team interaction within the same environment as the project's workspace, instead of keeping separated external tools; this is one of the main goals of the new system.

4.2.4 PERSISTENT COLLABORATIVE WORKSPACE PACKAGE

This package represents the persistence for a ColabY project's development environment. Instead of a fleeting coding session depending solely upon a live host, the envisioned workspace remains tied to the project, designed to sustain and maintain the developmental state, allowing individuals to depart from their workspace and return to it

to continue working later. This notion thus extends collaboration beyond a single meeting with the context carrying through over several such interactions.

1. **Workspace Management Module:** This module will be in charge of creating and organizing the development workspace related to the ColabY project. In addition to holding and cataloging various data resources, the workspace acts as a shared environment which manages project files, development resources, session information, and the collaborative state of the system.
2. **Persistent Session Module:** Instead of viewing collaborative sessions purely as connections between interactive users, this module can now categorize them as resources bound to the project. The "leaving, rejoining" behavior then allows the workspace concept to persist along sessions as individuals can enter, depart, and re-enter the shared session.
3. **File and Project State Module:** This module, at the core of persistence, concerns the retention of project files, directory layout, source code, configurations, and all data necessary to continue development later.
4. **Environment Management Module:** This module is responsible for managing the ColabY project's development environment in such a way as to maintain the consistency of its runtime and dependencies.
5. **Containerized Environment Module:** Through the usage of Docker environments for ColabY projects, the context retains the necessary associated configurations of dependencies, runtime, and other supporting applications.
6. **Workspace Resume Module:** This module is responsible for re-accessing the previous project session/workspace so that users can begin right where they left off.

These constitute a change from a transient real-time session based, collaborative event and towards more of a continuous/persistent ColabY development environment. Here, the project specifically recognizes that the host-dependent nature of the session and its consequent inability to re-initiate later is a key limitation which the idea of a session bound to the project can mediate.

4.2.5 REAL-TIME CODE SYNCHRONIZATION PACKAGE

This package is a main technical component in the ColabY collaborative programming environment that allows several authorized developers to work on the same development workspace and modify the same files together at the same time. It provides mechanisms to distinguish among tasks of editing, real-time communication and synchronous state update respectively. This package relies on Eclipse Theia for providing the development workspace, on WebSocket for delivering real-time transport, and on Yjs-based CRDT synchronization mechanism for handling shared collaborative state.

1. Collaborative IDE module – This module provides the development workspace through Eclipse Theia. Its responsibility is to facilitate programmer to edit source codes and interact with development workspace instead of building a full IDE from scratch for the ColabY.
2. Collaboration Session module – This module is responsible for maintaining active collaboration session and authorized project members in shared development workspace.
3. Real-Time Communication module – This module provides a channel of transmitting collaborative updating events among clients. It uses WebSocket to create a real-time event flow.
4. CRDT Synchronization module – this module represents shared collaborative states in CRDT format and manipulates the concurrent modifications that happen with different clients by using Yjs as the specified CRDT technique in ColabY
5. Document Synchronization module – This module keeps the state in the shared documents to sync among collaborators, when one client make changes in shared document, those changes will be delivered to other connected participants such that they will approach the same document state as well.
6. Presence and Awareness module –This module is responsible for keeping collaboration states which may involve whether a user is present within development workspace or not for other collaborators to monitor it.
7. Concurrent Editing module-this module synchronizes among concurrently modifying changes from different client to make them merge into shared collaborative state.

It contributes to let the ColabY provides active multi-users' coding instead of mere screen-sharing type development environment that allow the other collaborators to only observe rather than code. It should be noticed that the division of task between Theia for providing the whole IDE and WS, CRDT/Yjs for carrying the transporting task are important features for our intended architecture.

4.2.6 INTEGRATION AND ACCESS CONTROL PACKAGE

These packages provide the necessary backend support that ties the core ColabY services together and restricts access to project and collaboration resources to valid developers. Since ColabY manages user identities, project memberships, source code, and developer environments it needs robust authentication and authorization mechanisms which are provided by the following components:

1. Authentication Module- Verifies the identity of users prior to allowing access to secure resources that are available in ColabY.
2. Authorization Module- The module assigns privileges to users determining which operations and resources the users are allowed to access on the basis of project membership and assigned roles.
3. API Gateway Module- This module serves as the secure central point of access between the front-end, and backend services to manage requests and integrate to the APIs available on the back end.
4. Service Integration Module- Allows the backend services that deal with users, projects, and communications and collaboration modules to interact with each other.
5. Workspace Access Control Module- Assigns role based project level permissions to enable the controlled access of project workspaces by project collaborators.

This package acts as the bridge that ties the identity tier, the project tier and the collaboration tier, which ensures not only normal web API authentication/authorization, but more importantly in this case, authentication and authorization of project access for the coding workspace environment; indeed participants access control is described in the specification as critical to a collaborative coding space.

CHAPTER 5

IMPLEMENTATION RESULTS

5.1 IMPLEMENTATION RESULTS

The integration of ColabY The goal of the integration of ColabY is to achieve the important features of the proposed collaborative developer platform. The integrated system is able to manage developer profiles, facilitate developer search, enable project collaboration, support interaction within shared workspaces and host real-time collaborative development. These subsections are to describe the integrated system interfaces and the implemented features in detail..

5.1.1 IMPLEMENTATION OF DEVELOPER PROFILE

The developer profile module offers the users a way of creating and maintaining their personal developer identity in the ColabY platform. Profile page The profile page shows the information of the developer, indicating whether they are open to collaboration, their technical skills and the links to their external professional information. The sample implementation profile shown in Figure 5.1 shows the developer name and user ID, some short bio, the Open to Collaborate flag, and the skills of Java, React, Node and Spring Boot. The links to the GitHub and LinkedIn accounts are also included here to allow the other users to see the external technical background of the developer.

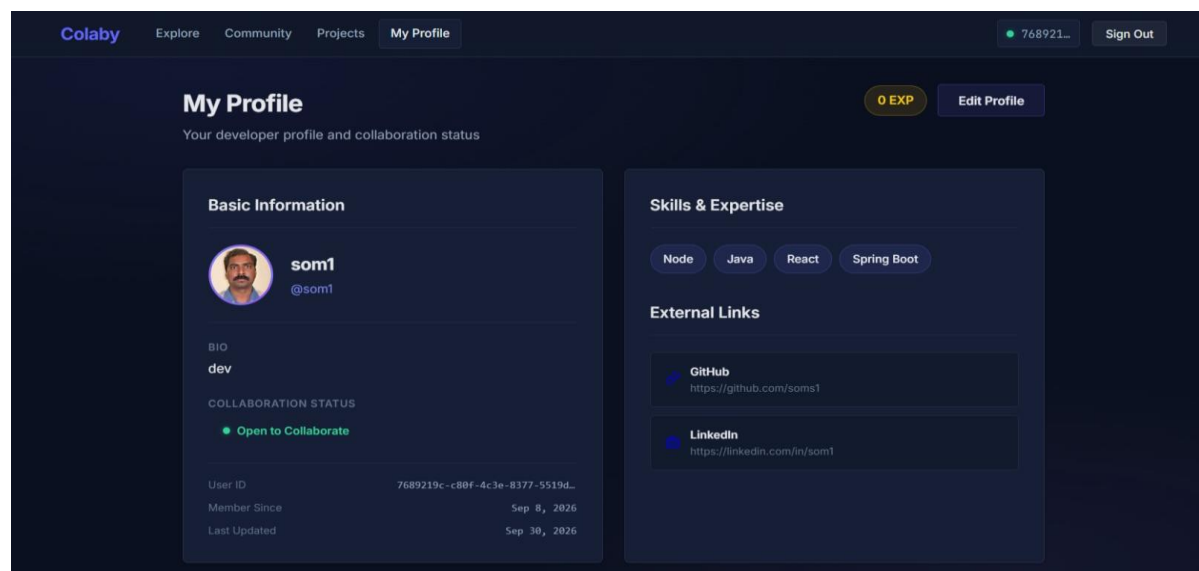


Figure 5.1 My Profile Interface

5.1.2 IMPLEMENTATION OF DEVELOPER DISCOVERY

The developer discovery module is used by the site users to find and study other developers in the site database and choose possible collaborators. On the Explore Developers page (Figure 5.2), users can perform a search by name, skill, username, or keyword. In addition, an Open to Collaborate filter helps locate developers available for collaboration. The search results offer the basic information about the developer – his profile, experience indicator, whether he is open to collaborate, and a view detailed button. This feature allows the initial discovery phase of developers based on their technical expertise and experience level.

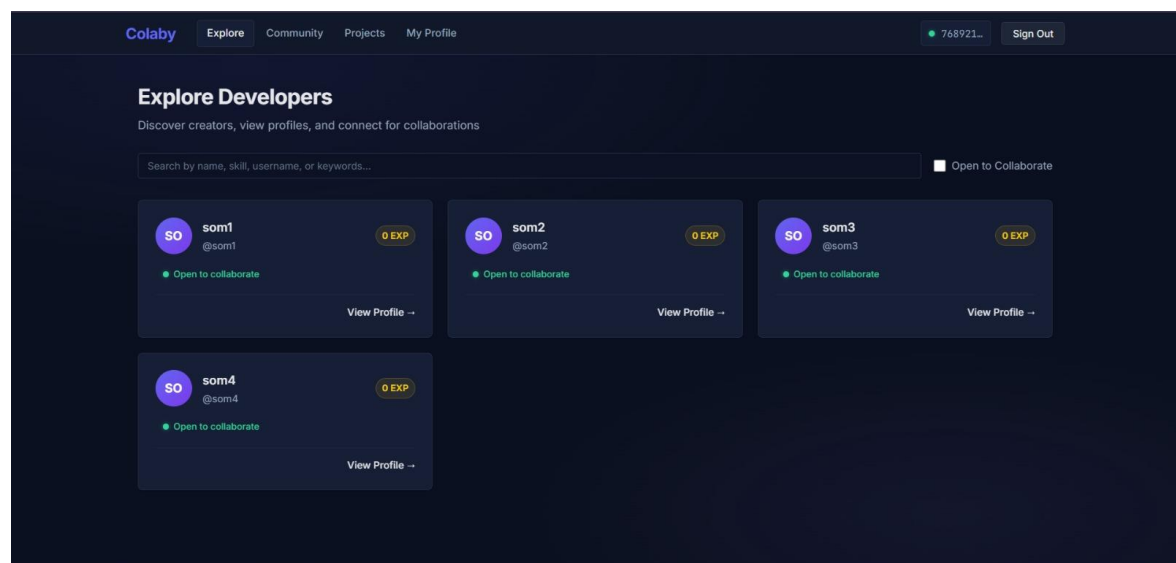


Figure 5.2 Developer Discovery and Search Interface

5.1.3 IMPLEMENTATION OF PROJECT COLLABORATION AND MEMBER MANAGEMENT

The project module allows for managing the members and collaboration requests. Figure 5.3 below shows the project interface which contains the project status, membership information and the option to open the project workspace. The interface allows for the Invite someone by user ID facility, which enables project owners to invite other developers to participate in the project. This feature also displays the pending join requests and provides the options for the project owner to accept or decline the request. This is the mechanism which would facilitate team formation and controlled participation in the collaborative projects.

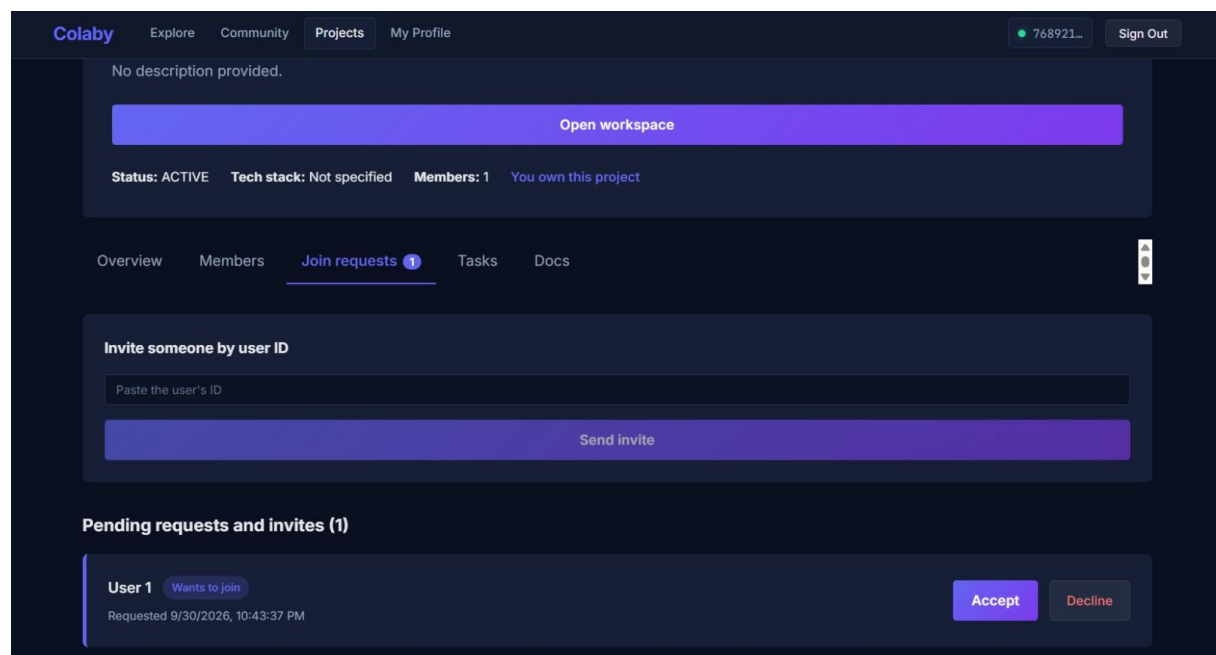


Figure 5.3 Project Collaboration and Member Management Interface

5.1.4 IMPLEMENTATION OF REAL-TIME COLLABORATIVE WHITEBOARD

Collaborative workspace also provides real-time drawing context for visual interaction between project members. Figure 5.4 below shows the collaborative canvas interface being accessed by more than one user. The canvas includes tools such as pen, shapes, lines, text, and eraser, allowing participants to create and modify drawings collaboratively. The same canvas state is visible to the connected participants, showcasing the use of the collaborative workspace for synchronized visual interaction. The tool can be used for activities such as brainstorming, architecture discussion, flow diagrams, and collaborative planning during software development.

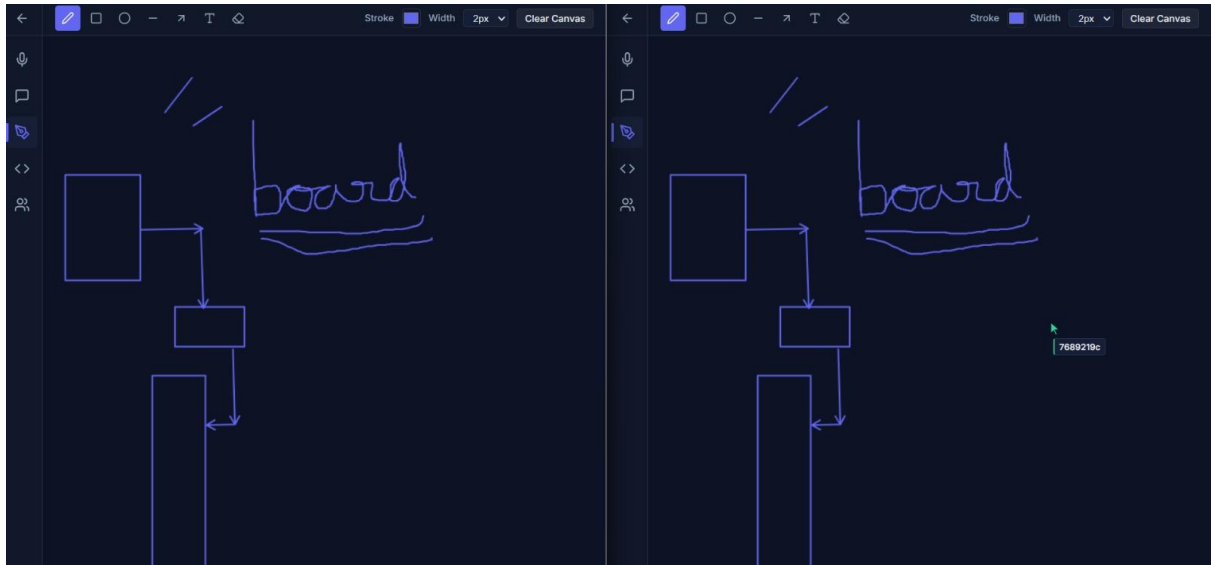


Figure 5.4 Real-Time Collaborative Whiteboard Interface

5.1.5 IMPLEMENTATION OF REAL-TIME COLLABORATIVE IDE

The real-time collaborative IDE is the main development component of ColabY. Figure 5.5 below represents the Eclipse Theia-based environment opened in two participants' browsers at the same time. The interface includes a project file explorer, source-code editor, terminal, and development tools at the bottom and top right area respectively. Top left area indicates an active real-time collaboration session and the connected peer. The screenshot represents one participant's area while another browser window is opened for another participant. Both are working on the same project workspace which implies the multi-user nature of the real-time collaboration feature. It can also be seen that real-time changes are made within the workspace area where participants actively collaborate, implying that developers can work on the same project instead of sharing a screen.

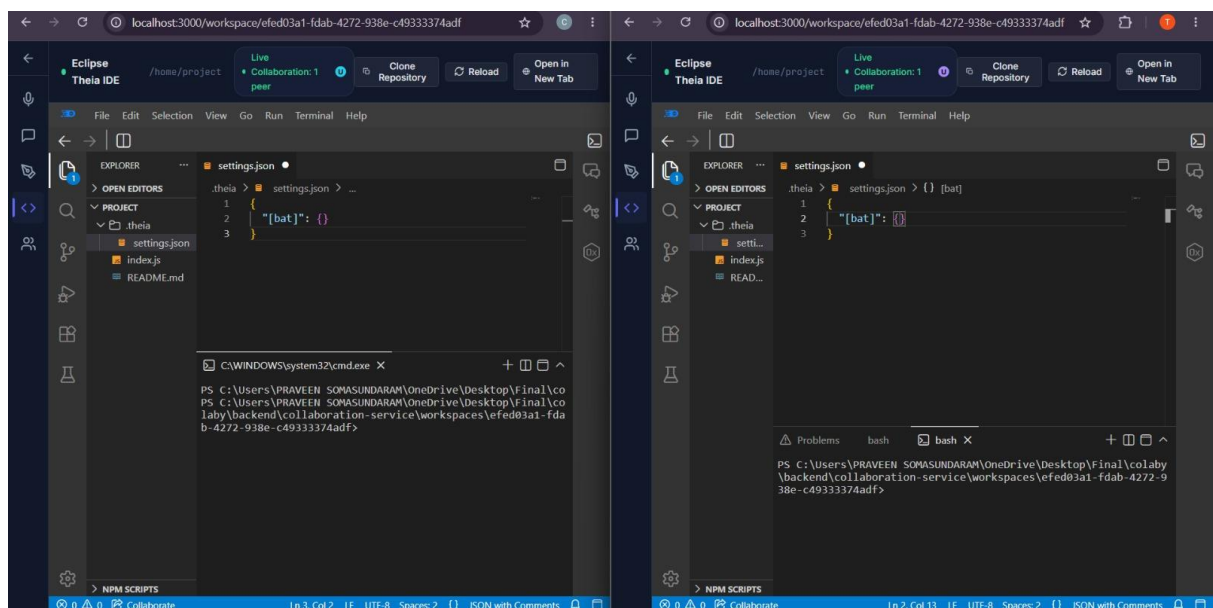


Figure 5.5 Real-Time Collaborative IDE

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

This proposal describes ColabY, an integrated web-based collaborative software development platform that bridges current software development workflow Fragmentation. The proposed system integrates: developer profiles, skill- & interest-based developer discovery, communities, project creation, project management, group communication, and real-time collaborative development environments in one place, helping developers less need to use external disconnected applications to find new development partners, organize projects, communicate with teammates, or perform development activities.

The real-time collaborative coding environment is the largest technical feature of the proposed ColabY system. The Eclipse Theia is used as the integrated development environment and WebSocket protocol for communication is employed for real-time updates of collaborative statuses while CRDTs (via Yjs) manages simultaneous concurrent edits of shared resources among developers. By this, we allow more than one developer to have effective contribution in working set of code, instead of watching over one another's screen.

Another characteristic design aspect of ColabY is the persistent development environment concept. In existing Collaborative coding and development platforms, collaboration is typically designed as short-term sessions and are host-dependent. But the projects within the system itself manage development environment with projects, helping developers disconnect and continue to their collaboration after long period break. The isolated development environment also is a good basis for per-project environment configuration.

ColabY establishes an all-in-one platform that connects the social and organizational aspects with the technical collaboration to improve software development process. Using developer discovery, team building, project and group management, effective team communication, persistence features, as well as CRDTs and collaborative development, ColabY has proven its ability to contribute to a seamless and continuous software development environment.

.

6.2 FUTURE WORK

In the future, ColabY can move forward to explore scaling, improved usability, enhanced developer capabilities of the described collaborative workspace and more. Further, with respect to the concept of a persistent workspace, more thorough preservation of the project state can be achieved to maintain the development context across sessions; also, for collaborative development, IDEs, language tools, debugging support and other development workflows can be integrated into the collaborative development environment.

More optimization in terms of scalability regarding concurrent users and number of concurrent sessions will be conducted on real-time collaboration layer. More effort will be contributed to the CRDT based synchronizations and WebSocket infrastructures to speed up synchronization, maintain connection and reliability for larger loads. Different stack for projects running in container based project environments will be considered by the system.

In addition, the developers' discovery can be strengthened in future versions to create matching for developers in accordance with interests, skills and project demands. Knowledge sharing among developers and the skill enhancement features on its community can be developed to further grow developer communities. Stronger linkage between long-term project version control workflow and real-time collaborative editing work can be achieved using version control & repository management integrations.

The whole system will be further deployed, scaled, and tested in real-life Collaborative Software Development scenarios, together with its evaluation and comparisons to existing works.

REFERENCES

- [1] Ömer Esas. *Design Patterns and Anti-Patterns in Microservices Architecture: A Classification Proposal and Study on Open Source Projects*. Master's thesis, Politecnico di Milano, 2022.
- [2] Anastasiia Tkalic, Nils Brede Moe, Nina Haugland Andersen, Viktoria Stray, and Astri Moksnes Barbala. *Pair Programming Practiced in Hybrid Work*. In *Proceedings of the 2023 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, pages 1–7, 2023. doi: 10.1109/ESEM56168.2023.10304797.
- [3] Gleiph Ghiotto, Leonardo Murta, Márcio Barros, and André van der Hoek. *On the Nature of Merge Conflicts: A Study of 2,731 Open Source Java Projects Hosted by GitHub*. *IEEE Transactions on Software Engineering*, vol. 46, no. 8, pages 892–915, 2020. doi: 10.1109/TSE.2018.2871083.
- [4] Prabhnoor Singh, Adesh Mishra, and Jasleen Kaur. *Real Time Collaborative Whiteboard*. *International Journal for Research in Applied Science & Engineering Technology*, vol. 12, no. 11, pages 431–436, 2024. doi: 10.22214/ijraset.2024.64889.
- [5] Bogdan Vasilescu, Alexander Serebrenik, Premkumar Devanbu, and Vladimir Filkov. *How Social Q&A Sites are Changing Knowledge Sharing in Open Source Software Communities*. In *Proceedings of the 17th ACM Conference on Computer Supported Cooperative Work and Social Computing (CSCW 2014)*, pages 342–354, 2014. doi: 10.1145/2531602.2531659.
- [6] Anders Sundelin, Javier Gonzalez-Huerta, Richard Torkar, and Krzysztof Wnuk. *Governing the Commons: Code Ownership and Code-Clones in Large-Scale Software Development*. *Empirical Software Engineering*, vol. 30, article 43, 2025. doi: 10.1007/s10664-024-10598-7.
- [7] Hitesh Sapkota, Pradeep K. Murukannaiah, and Yi Wang. *A Network-Centric Approach for Estimating Trust Between Open Source Software Developers*. *PLOS ONE*, vol. 14, no. 12, article e0226281, 2019. doi: 10.1371/journal.pone.0226281.
- [8] S. Steffi Nivedita, P. U. Harsha, P. Rahul, P. Surya Tej, and P. Venkat Balaji Naidu. *A Unified Multi-Modal Real-Time Collaborative Development Environment Integrating Code Editing, Voice Communication, Drawing, and File Synchronization*. *International Journal of Scientific Research and Technology*, vol. 2, no. 12, pages 385–390, 2025. doi: 10.5281/zenodo.18048315.
- [9] Xin Tan, Xinyue Lv, Jing Jiang, and Li Zhang. *Understanding Real-Time Collaborative Programming: A Study of Visual Studio Live Share*. *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 4, article 110, 2024. doi: 10.1145/3643672.

[10] Aparna A. Veer, Omkar Mane, Prathamesh Jadhav, and Atharvi Chevale. *A Comprehensive Study on Real-Time Web IDE Collaborative Code Editors. International Journal for Research in Applied Science & Engineering Technology*, vol. 13, no. 11, pages 1227–1234, 2025. doi: 10.22214/ijraset.2025.75276